ENGINEERING PRD & SYSTEMS ARCHITECTURE

ScreenForge AI: Text-Driven Video Recorder & Editor

Author: Lead Architect | **Status:** Approved for MVP | **Target:** Web MVP → Desktop Shell | **Stack:** Web Media APIs + Groq/Together LPU Fallback + FFmpeg

Executive Abstract: ScreenForge is an agile, low-latency alternative to Loom engineered for technical professionals. It pairs zero-overhead web screen capture with synchronized word-level transcription. Instead of cutting video on a complex timeline, users edit their spoken script directly in text. Changes trigger non-destructive instant playback via playhead skipping and headless FFmpeg concatenation upon final export.

1. Target Problems & Core Value Proposition

Current Friction (Loom / Traditional)	ScreenForge Engineering Solution	Business & Productivity Impact
Heavy client footprint and sluggish timeline manipulation.	Native in-browser capture with instant DOM-level transcript jumping.	Zero installation required; immediate recording launch under 3 seconds.
Tedious timeline slicing for false starts, verbal ticks, and silences.	Whisper word-stamps + automated filler detection via multi-provider LLM.	Rough-cut video produced in seconds with zero manual split actions.
Massive media uploads causing API timeout limits and high latency.	Client-side Opus downsampling (16kHz mono) separates audio pipeline.	Reduces transmission payload by >90%, avoiding 25MB HTTP upload ceilings.
Single-provider API failure (rate caps, 429 errors, downtime).	Multi-provider fallback engine (Groq → Cerebras → Together AI).	99.9% ingestion reliability without throttling or dropped operations.

2. High-Level Technology Stack & Responsibilities

Client Layer (Frontend) Zero Bloat
Stack: Next.js (React), Tailwind CSS, Web Audio API, MediaRecorder API.
Responsibilities: Manages screen/mic streams, provides word-synced highlight text player, and renders instant skips without client rendering overhead.

AI Inference Pipeline Open Models
Stack: Whisper Large v3 (via Groq/Together) + LLaMA 3.3 70B Versatile.
Responsibilities: Word-level alignment timestamps, smart silence detection, filler elimination recommendations, and chapter generation.

Backend Processing Service High Concurrency
Stack: Python FastAPI, Uvicorn, FFmpeg Core Engine.
Responsibilities: Multi-provider failover routing, video payload storage, precise non-destructive trim execution, and lossless MP4 rendering.

Desktop Evolution Layer Phase 2 Target
Stack: Tauri (Rust) wrapper over the production Next.js frontend.
Responsibilities: Native OS capture hooks, system tray minimization, and global hotkeys (`Cmd+Shift+R`) with negligible RAM impact.

3. Product Functional Requirements

- **Atomic Stream Ingestion:** Record screen video and mic voice into a single multiplexed container to maintain hardware-clock synchrony (PTS zero drift).
- **Synchronized Transcript Grid:** Display clickable transcript tokens bound to video time indices. Clicking a word scrubs the video immediately to that second.
- **Transcript-Driven Trimming:** Selecting and striking through transcript text flags corresponding time ranges for excision in both audio and video streams.
- **Automated Silence and Filler Removal:** Auto-highlight dead air >1.5s and common disfluencies with a 1-click batch accept function.

4. End-to-End Operational Workflow (Step-by-Step)



5. Provider Reliability & Fallback Matrix

Tier Priority	Inference Provider	Model Assigned	Role & Failover Condition
Tier 1 (Primary)	**Groq Cloud**	Whisper v3 + LLaMA 3.3 70B	Primary route. Sub-second generation. Fails over on 429, 413, or 503 HTTP status.
Tier 2 (Failover)	**Cerebras Cloud**	LLaMA 3.3 70B	Activated on Groq LLM rate-limit; handles immediate script structuring at 1000+ t/s.
Tier 3 (Failover)	**Together AI**	Whisper Large v3 + LLaMA 3.3	Enterprise SLA backup if Tier 1 and 2 experience degraded capacity or regional drops.
Local Offline Fallback	**Local In-Memory**	RegEx Rules Engine	Runs in browser/client: cleans common filler words instantly without API calls.

6. Non-Destructive Skip vs. Final Export Architecture

To deliver real-time responsiveness without re-encoding delays, video editing occurs in two distinct environments:

```
// 1. Client-Side Instant Playback (Zero Processing Latency) videoElement.ontimeupdate = () => { const activeCut = cutRanges.find(cut => video.currentTime >= cut.start && video.currentTime < cut.end); if (activeCut) { video.currentTime = activeCut.end; } // Playhead teleportation }; // 2. Final Server-Side Headless Cut (FFmpeg Concat Filter) ffmpeg -i input.webm -filter_complex \ "[0:v]trim=0.00:1.20,setpts=PTS-STARTPTS[v0]; [0:a]atrim=0.00:1.20,asetpts=PTS-STARTPTS[a0]; \ [0:v]trim=1.55:14.2,setpts=PTS-STARTPTS[v1]; [0:a]atrim=1.55:14.2,asetpts=PTS-STARTPTS[a1]; \ [v0][a0][v1][a1]concat=n=2:v=1:a=1[outv][outa]" \ -map "[outv]" -map "[outa]" -c:v libx264 -crf 20 -c:a aac export_clean.mp4
```

7. Production AI Prompts & Schema Specifications

Prompt 1: LLaMA 3.3 Cut Planner JSON Mode

System Instruction:

```
You are an expert video editor. You receive a transcript array with word timestamps: [{word, start, end}]. Identify: 1. Verbal disfluencies ("um", "uh", "you know", stuttered repeats). 2. Dead air gaps > 1.5 seconds. 3. False starts (speaker abandons a sentence mid-phrase). Return ONLY JSON: { "cuts": [ {"start": float, "end": float, "words": string, "reason": "filler" | "silence" | "false_start"} ] } Rule: Never invent time ranges. Every start and end must strictly match indices in the input array.
```

Prompt 2: Summary & Chapter Generator Post-Process

System Instruction:

```
You are a technical documentarian. Using the clean transcript with time markers, generate: 1. summary: A 2-sentence executive summary. 2. action_items: Bullet points of explicit follow-ups or bugs mentioned. 3. chapters: Array of [{title: string, timestamp: float}] marking major semantic transitions. Return ONLY valid JSON matching this schema: { "summary": string, "action_items": string[], "chapters": [{"title": string, "timestamp": float}] }
```

8. Technical Implementation Milestones